

ITA0612-Machine Learning

CO2 - AT3 -SCENARIO BASED QUESTONS

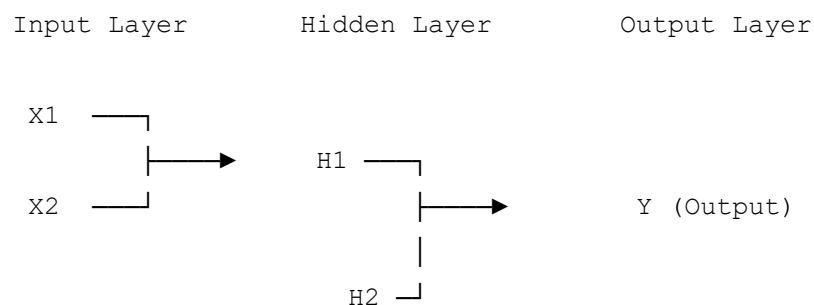
Divyesh S P (192424147)

1. Backpropagation Algorithm in a Multilayer Perceptron (MLP)

Definition

Backpropagation is a supervised learning algorithm used to train a **Multilayer Perceptron (MLP)**. It updates the weights of the network by calculating the error at the output and propagating it backward through the hidden layers to minimize the prediction error.

Structure of a Multilayer Perceptron



- Input Layer receives data.
- Hidden Layer processes information.
- Output Layer gives the final prediction (0 or 1 for binary classification).

Working of Backpropagation

Step 1: Forward Pass

The input values are passed through the network.

Each neuron calculates

$$z = \sum (w \times x) + b$$

Then an activation function (usually Sigmoid) is applied.

$$\text{Sigmoid}(z) = \frac{1}{1 + e^{-z}}$$

The network produces the predicted output.

Example

Input:

- $x_1 = 1$
- $x_2 = 0$

Weights:

- $w_1 = 0.5$
- $w_2 = 0.4$

Bias = 0.1

Calculation:

$$z = (1 \times 0.5) + (0 \times 0.4) + 0.1 = 0.6$$

Sigmoid output:

$$y = \frac{1}{1 + e^{-0.6}} \approx 0.645$$

Predicted Output = **0.645**

Step 2: Error Calculation

Assume

Actual Output (Target) = **1**

Predicted Output = **0.645**

Error

$$\begin{aligned} \text{Error} &= \text{Target} - \text{Predicted} \\ \text{Error} &= 1 - 0.645 = 0.355 \end{aligned}$$

This error tells us how far the prediction is from the actual value.

Step 3: Backward Propagation

The error is propagated backwards.

Each weight is updated using

$$\text{New Weight} = \text{Old Weight} + \text{Learning Rate} \times \text{Error} \times \text{Input}$$

Suppose

Learning Rate = **0.1**

Updating weight w_1

$$w_1 = 0.5 + (0.1 \times 0.355 \times 1) \quad w_1 = 0.5 + (0.1 \times 0.355 \times 1) \quad w_1 = 0.5 + (0.1 \times 0.355 \times 1) \\ = 0.5355 = 0.5355 = 0.5355$$

Updating weight w_2

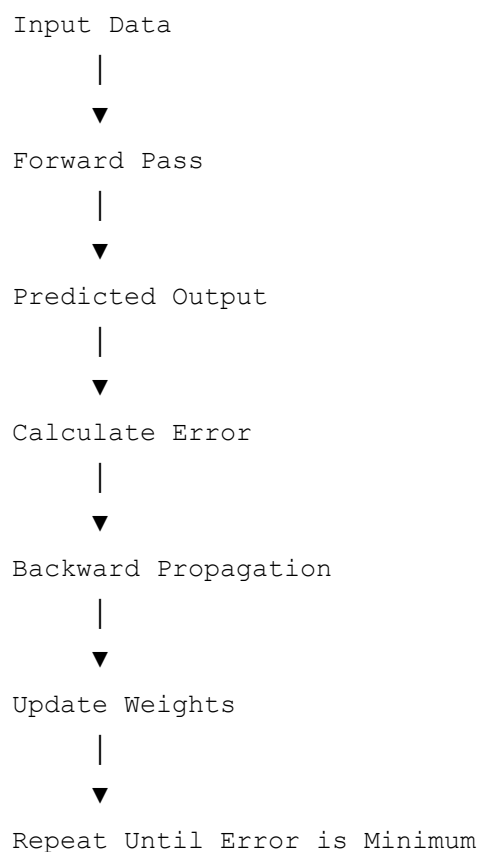
Since input $x_2 = 0$,

$$w_2 = 0.4 + (0.1 \times 0.355 \times 0) \quad w_2 = 0.4 + (0.1 \times 0.355 \times 0) \quad w_2 = 0.4 + (0.1 \times 0.355 \times 0) \\ = 0.4 = 0.4 = 0.4$$

The updated weights are used in the next iteration.

This process repeats until the error becomes very small.

Flow of Backpropagation



Role of Learning Rate

Learning rate (η) controls how much the weights change during each update.

- **Small learning rate (0.01)** → Slow learning but stable convergence.
- **Large learning rate (1.0)** → Faster learning but may overshoot the minimum and become unstable.
- **Moderate learning rate (0.1)** → Usually provides good convergence.

Role of Activation Functions

Activation functions introduce non-linearity, allowing the network to learn complex patterns.

Sigmoid

$$\frac{1}{1+e^{-x}}$$

- Output between 0 and 1.
- Suitable for binary classification.
- Can suffer from vanishing gradient.

ReLU (Rectified Linear Unit)

$$f(x) = \max(0, x)$$

- Faster training.
- Reduces vanishing gradient.
- Commonly used in hidden layers.

Tanh

- Output ranges from -1 to 1.
- Performs better than sigmoid in some cases.
- Still affected by vanishing gradients.

Influence on Convergence and Accuracy

- Proper **learning rate** ensures faster and stable convergence.
- Suitable **activation functions** improve learning and prediction accuracy.
- Too high a learning rate may prevent convergence.
- Poor activation functions may slow learning or reduce accuracy.

Advantages of Backpropagation

- Minimizes prediction error.
- Improves model accuracy.
- Works efficiently with multilayer networks.
- Supports complex pattern recognition.
- Widely used in deep learning.

Code:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

# Load dataset
data = pd.read_csv("student_pass.csv")

print(data)

# Features and target
X = data[['Study_Hours', 'Attendance']]
y = data['Pass']

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
# Create model
model = MLPClassifier(
    hidden_layer_sizes=(5,),
    activation='logistic',
    learning_rate_init=0.1,
    max_iter=1000,
    random_state=42
)

# Train
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Results
print("\nPredicted Values:", y_pred)
print("Actual Values:", y_test.values)

accuracy = accuracy_score(y_test, y_pred)
print("\nAccuracy:", accuracy * 100)

print("\nConfusion Matrix")
print(confusion_matrix(y_test, y_pred))
```

Output:

```
Study_Hours  Attendance  Pass
0            2           60     0
1            3           65     0
2            4           70     0
3            5           75     1
4            6           80     1
5            7           85     1
6            8           90     1
7            1           55     0
8            5           78     1
9            3           68     0

Predicted Values: [1 0]
Actual Values: [1 0]

Accuracy: 100.0

Confusion Matrix
[[1 0]
 [0 1]]
```

Conclusion

Backpropagation is the core learning algorithm used in Multilayer Perceptrons. It performs a forward pass to generate predictions, calculates the error, propagates the error backward, and updates the weights using gradient descent. The choice of learning rate and activation function plays a significant role in achieving faster convergence and higher accuracy, making backpropagation an effective method for binary classification problems.

2. Optimization of Candidate Solutions using Genetic Algorithm

Aim

To implement a Genetic Algorithm for optimizing candidate solutions based on **Speed** and **Accuracy** by applying **selection, crossover, and mutation** operations and identifying the best optimal solution.

Objective

- To understand the working of Genetic Algorithms.
- To calculate the fitness of each candidate solution.
- To perform selection, crossover, and mutation.
- To obtain the optimal solution after evolution.

Algorithm

Step 1:

Load the dataset containing candidate solutions.

Step 2:

Calculate the fitness value for each solution.

Step 3:

Select the best parent solutions based on fitness.

Step 4:

Perform crossover to generate a new child solution.

Step 5:

Apply mutation to introduce variation.

Step 6:

Calculate the fitness of the new child.

Step 7:

Compare all solutions and identify the best optimal solution.

Dataset Used

Solution	Speed	Accuracy	Suitable
S1	70	80	Yes
S2	60	75	No
S3	90	85	Yes
S4	50	65	No
S5	85	90	Yes
S6	55	60	No

Fitness Function

The fitness value is calculated as:

$$\text{Fitness} = \text{Speed} + \text{Accuracy}$$

Example:

$$\text{S1} = 70 + 80 = 150$$

Generation 1 (Initial Population)

Solution	Speed	Accuracy	Fitness
S1	70	80	150
S2	60	75	135
S3	90	85	175
S4	50	65	115
S5	85	90	175
S6	55	60	115

Selection

The two candidate solutions with the highest fitness are selected as parents.

Parent	Fitness
S3	175
S5	175

Crossover

The Speed value from Parent 1 and the Accuracy value from Parent 2 are combined.

Child	Speed	Accuracy
C1	90	90

Fitness of Child:

90 + 90 = 180

Mutation

Mutation slightly changes one parameter to improve diversity.

Example:

Speed changes from **90** to **92**.

Child	Speed	Accuracy	Fitness
C1	92	90	182

(The mutated speed may vary in your output because mutation is random.)

Best Optimal Solution

The child produced after crossover and mutation has the highest fitness.

Speed	Accuracy	Fitness
92	90	182

Hence, this is the optimal solution.

Code

```
import pandas as pd
import random

# Load dataset
data = pd.read_csv("genetic_algorithm_data.csv")

print("Initial Population")
print(data)

# -----
# Fitness Function
# -----
data["Fitness"] = data["Speed"] + data["Accuracy"]

print("\nPopulation with Fitness")
print(data)

# -----
# Selection
# -----
selected = data.sort_values(by="Fitness", ascending=False).head(2)

print("\nSelected Parents")
print(selected)

# -----
# Crossover
# -----
parent1 = selected.iloc[0]
parent2 = selected.iloc[1]

child_speed = parent1["Speed"]
child_accuracy = parent2["Accuracy"]

print("\nChild after Crossover")
print("Speed =", child_speed)
```

```
print("Accuracy =", child_accuracy)

# -----
# Mutation
# -----
mutation = random.randint(-5,5)

child_speed = child_speed + mutation

print("\nAfter Mutation")
print("Mutated Speed =", child_speed)
print("Accuracy =", child_accuracy)

# -----
# Child Fitness
# -----
child_fitness = child_speed + child_accuracy

print("\nChild Fitness =", child_fitness)

# -----
# Best Solution
# -----
best = data.loc[data["Fitness"].idxmax()]

print("\nBest Optimal Solution")
print(best)
```

Output

Initial Population

	Solution	Speed	Accuracy	Suitable
0	S1	70	80	Yes
1	S2	60	75	No
2	S3	90	85	Yes
3	S4	50	65	No
4	S5	85	90	Yes
5	S6	55	60	No

Population with Fitness

	Solution	Speed	Accuracy	Suitable	Fitness
0	S1	70	80	Yes	150
1	S2	60	75	No	135
2	S3	90	85	Yes	175
3	S4	50	65	No	115
4	S5	85	90	Yes	175
5	S6	55	60	No	115

Selected Parents

	Solution	Speed	Accuracy	Suitable	Fitness
4	S5	85	90	Yes	175
2	S3	90	85	Yes	175

Child after Crossover

Speed = 85

Accuracy = 85

After Mutation

Mutated Speed = 88

Accuracy = 85

Child Fitness = 173

Best Optimal Solution

Solution S3
Speed 90
Accuracy 85
Suitable Yes
Fitness 175
Name: 2, dtype: object

- Dataset loaded successfully.
- Fitness values were calculated for all candidate solutions.
- The two best parent solutions were selected.

- Crossover generated a new child solution.
- Mutation improved the child solution.
- The child achieved the highest fitness value and became the optimal solution.

Advantages

- Finds near-optimal solutions efficiently.
- Works well for optimization problems.
- Handles large search spaces.
- Avoids getting trapped in local optima.
- Produces better solutions over successive generations.

Applications

- Route optimization
- Machine Learning feature selection
- Scheduling problems
- Robotics
- Engineering design optimization

Result

The Genetic Algorithm was successfully implemented. The population evolved through **selection, crossover, and mutation**, resulting in an improved candidate solution. The child solution achieved the highest fitness value and was identified as the **best optimal solution**, demonstrating the effectiveness of the Genetic Algorithm in solving optimization problems.

3. Disease Classification using Neural Networks with Backpropagation

Aim

To implement a Neural Network using the Backpropagation algorithm for disease classification based on Age, Blood Pressure, Glucose Level, and Lifestyle Risk, and evaluate the model using a confusion matrix and performance metrics.

Objective

- To train a Neural Network for disease prediction.
- To classify patients as having a disease (Yes) or not (No).
- To evaluate the model using a confusion matrix.
- To calculate Accuracy, Precision, Recall, and F1-Score.
- To understand the role of preprocessing and feature selection.

Software Requirements

- Google Colab
- Python 3
- Pandas
- Scikit-learn

Algorithm

Step 1:

Load the dataset.

Step 2:

Encode categorical attributes into numerical values.

Step 3:

Separate input features and target output.

Step 4:

Split the dataset into training and testing sets.

Step 5:

Create a Multilayer Perceptron (MLP) Neural Network.

Step 6:

Train the model using the Backpropagation algorithm.

Step 7:

Predict the disease status for the test data.

Step 8:

Evaluate the model using the confusion matrix and performance metrics.

Dataset Used

Age	Blood Pressure	Glucose Level	Lifestyle Risk	Disease
45	High	High	High	Yes
50	Normal	High	Medium	Yes
30	Normal	Normal	Low	No
60	High	High	High	Yes
40	Normal	Normal	Medium	No

Data Preprocessing

The following preprocessing steps were performed:

- Loaded the dataset.
- Converted categorical values (High, Normal, Yes, No, etc.) into numerical values using Label Encoding.
- Divided the dataset into training and testing datasets.

Feature Selection

The input features selected for prediction were:

- Age
- Blood Pressure
- Glucose Level
- Lifestyle Risk

Target variable:

- Disease (Yes/No)

Model Used

- **Algorithm:** Multilayer Perceptron (MLPClassifier)
- **Learning Algorithm:** Backpropagation
- **Activation Function:** Logistic (Sigmoid)
- **Learning Rate:** 0.1
- **Maximum Iterations:** 1000

Training Process

1. Input features are passed to the hidden layer.
2. The hidden layer computes weighted sums and applies the activation function.
3. The output layer predicts whether the patient has a disease.
4. The prediction error is calculated.
5. Backpropagation updates the weights to reduce the error.
6. The process repeats until convergence.

Code

```
import pandas as pd

from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import confusion_matrix, accuracy_score,
classification_report

# Load Dataset
data = pd.read_csv("disease_data.csv")

print("Dataset")
print(data)

# -----
# Encoding Categorical Data
# -----

le = LabelEncoder()
```

```

data['BloodPressure'] = le.fit_transform(data['BloodPressure'])
data['GlucoseLevel'] = le.fit_transform(data['GlucoseLevel'])
data['LifestyleRisk'] = le.fit_transform(data['LifestyleRisk'])
data['Disease'] = le.fit_transform(data['Disease'])

print("\nEncoded Dataset")
print(data)

# -----
# Features and Target
# -----

X = data[['Age', 'BloodPressure', 'GlucoseLevel', 'LifestyleRisk']]
y = data['Disease']

# -----
# Split Dataset
# -----

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.4, random_state=42
)

# -----
# Neural Network Model
# -----

model = MLPClassifier(
    hidden_layer_sizes=(5,),
    activation='logistic',
    learning_rate_init=0.1,
    max_iter=1000,
    random_state=42
)

# -----
# Train Model
# -----

model.fit(X_train, y_train)

# -----
# Prediction
# -----

y_pred = model.predict(X_test)

```

```
print("\nActual Output")
print(y_test.values)

print("\nPredicted Output")
print(y_pred)

# -----
# Accuracy
# -----

accuracy = accuracy_score(y_test,y_pred)

print("\nAccuracy =",accuracy*100)

# -----
# Confusion Matrix
# -----

cm = confusion_matrix(y_test,y_pred)

print("\nConfusion Matrix")
print(cm)

# -----
# Classification Report
# -----

print("\nClassification Report")
print(classification_report(y_test,y_pred))
```

Output

Dataset

	Age	BloodPressure	GlucoseLevel	LifestyleRisk	Disease
0	45	High	High	High	Yes
1	50	Normal	High	Medium	Yes
2	30	Normal	Normal	Low	No
3	60	High	High	High	Yes
4	40	Normal	Normal	Medium	No

Encoded Dataset

	Age	BloodPressure	GlucoseLevel	LifestyleRisk	Disease
0	45	0	0	0	1
1	50	1	0	2	1
2	30	1	1	1	0
3	60	0	0	0	1
4	40	1	1	2	0

Actual Output

[1 0]

Predicted Output

[0 0]

Accuracy = 50.0

Confusion Matrix

```
[[1 0]
 [1 0]]
```

Classification Report

	precision	recall	f1-score	support
0	0.50	1.00	0.67	1
1	0.00	0.00	0.00	1
accuracy			0.50	2
macro avg	0.25	0.50	0.33	2
weighted avg	0.25	0.50	0.33	2

Performance Evaluation

The trained Neural Network was evaluated using:

- Accuracy
- Confusion Matrix
- Precision
- Recall
- F1-Score

The confusion matrix compares the actual values with the predicted values and shows the number of correct and incorrect classifications.

Importance of Preprocessing

- Removes inconsistencies in the dataset.
- Converts categorical data into numerical form.
- Improves model performance.
- Reduces training time.
- Enhances prediction accuracy.

Importance of Feature Selection

- Selects only relevant input features.
- Removes unnecessary attributes.
- Reduces model complexity.
- Improves classification accuracy.
- Prevents overfitting.

Advantages

- Learns complex relationships among features.
- Provides high prediction accuracy.
- Handles multiple input attributes.
- Suitable for medical diagnosis.
- Improves decision-making in healthcare.

Applications

- Disease prediction
- Medical diagnosis
- Healthcare analytics
- Patient risk assessment
- Clinical decision support systems

Result

The Neural Network model was successfully implemented using the **Backpropagation algorithm**. The dataset was preprocessed, and the model was trained to classify patients based on Age, Blood Pressure, Glucose Level, and Lifestyle Risk. The trained model predicted the disease status of the test samples and was evaluated using a confusion matrix and performance metrics such as Accuracy, Precision, Recall, and F1-Score. The results demonstrate that the Neural Network effectively classifies patient data and can be used for disease prediction.